Combinational Logic - Multiplexers (MUX)

1. Introduction to Multiplexers

- A digital logic circuit that is capable of accepting several inputs and generating a single output is known as a multiplexer or MUX.
- The multiplexer functions as a data selector that takes many inputs and provides one selected output.
- In a multiplexer, there are 2^n input lines and 1 output line, where n represents the number of select lines.
- Because of this specific architecture, the multiplexer is commonly referred to as a "many-to-one" circuit.

Types of Multiplexers: Multiplexers are primarily classified into four types based on their input and select line configurations:

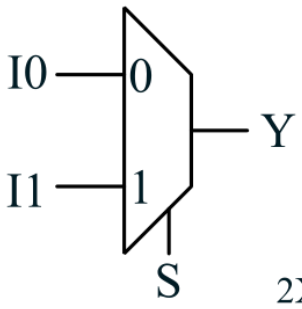
1. 2-to-1 multiplexer (1 select line)
2. 4-to-1 multiplexer (2 select lines)
3. 8-to-1 multiplexer (3 select lines)
4. 16-to-1 multiplexer (4 select lines)

2. The 2x1 Multiplexer

- The 2×1 MUX is a fundamental circuit used to choose one signal from two inputs and transmit it to the output.
- It consists of two input lines (I_0, I_1), one output line (Y), and a single selection line (S).

Truth Table:

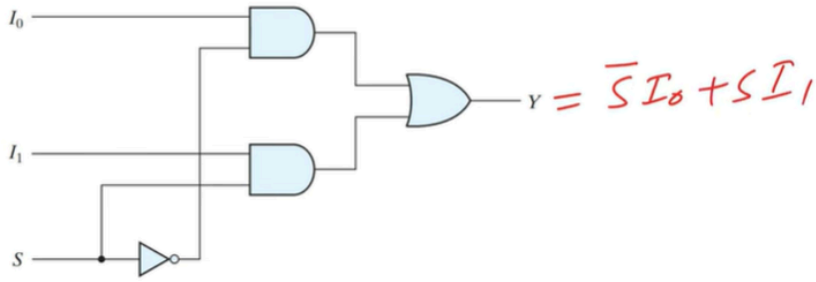
S (Select)	Y (Output)
0	I_0
1	I_1



2X1 MUX

Boolean Expression: When the select line S is logic 0, I_0 is selected, and when S is logic 1, I_1 is selected.

$$Y = \overline{S}I_0 + SI_1$$



3. The 4x1 Multiplexer

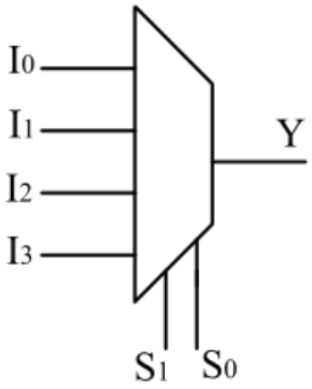
- A 4×1 MUX has four data inputs (I_0, I_1, I_2, I_3) and only one output (Y).
- It requires 2 select lines (S_1, S_0) for operation.

Logic Rules:

- When both select lines are logic 0 ($S_1 = 0, S_0 = 0$), I_0 is selected.
- When S_1 is logic 0 and S_0 is logic 1, I_1 is selected.
- When S_1 is logic 1 and S_0 is logic 0, I_2 is selected.
- When both select lines are logic 1 ($S_1 = 1, S_0 = 1$), I_3 is selected.

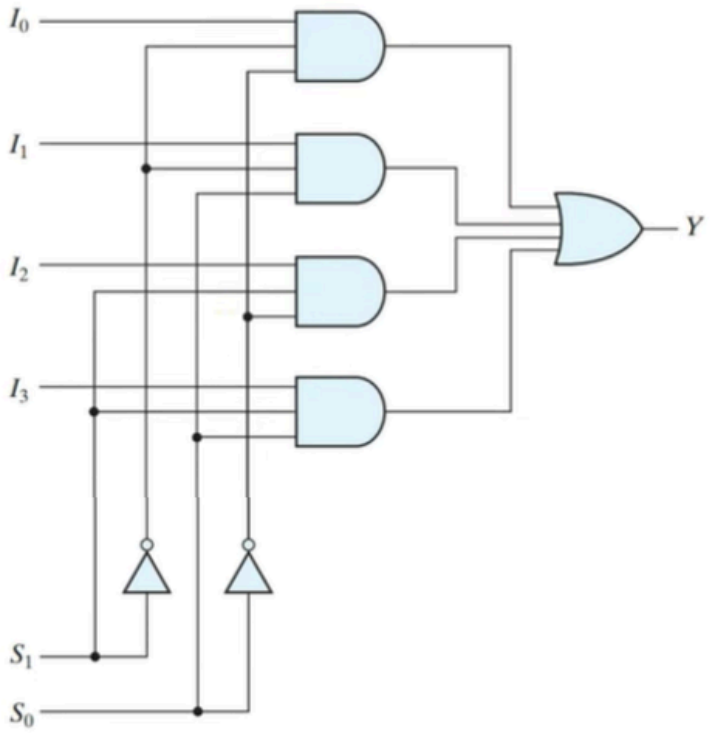
Truth Table:

S_1	S_0	Y (Output)
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



Boolean Expression:

$$Y = \overline{S}_1\overline{S}_0I_0 + \overline{S}_1S_0I_1 + S_1\overline{S}_0I_2 + S_1S_0I_3$$

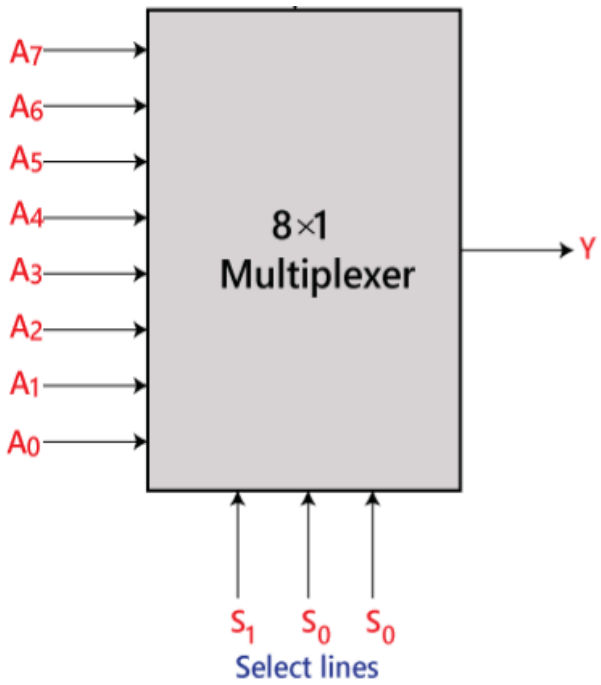

I_0
I_1
I_2
I_3
S_1
S_0
Y

4. The 8x1 Multiplexer

- In the 8×1 multiplexer, there are eight inputs (A_0 to A_7), 3 selection lines (S_2, S_1, S_0), and a single output (Y).
- Based on the binary combination present at the selection lines, one of the 8 inputs is connected to the output.

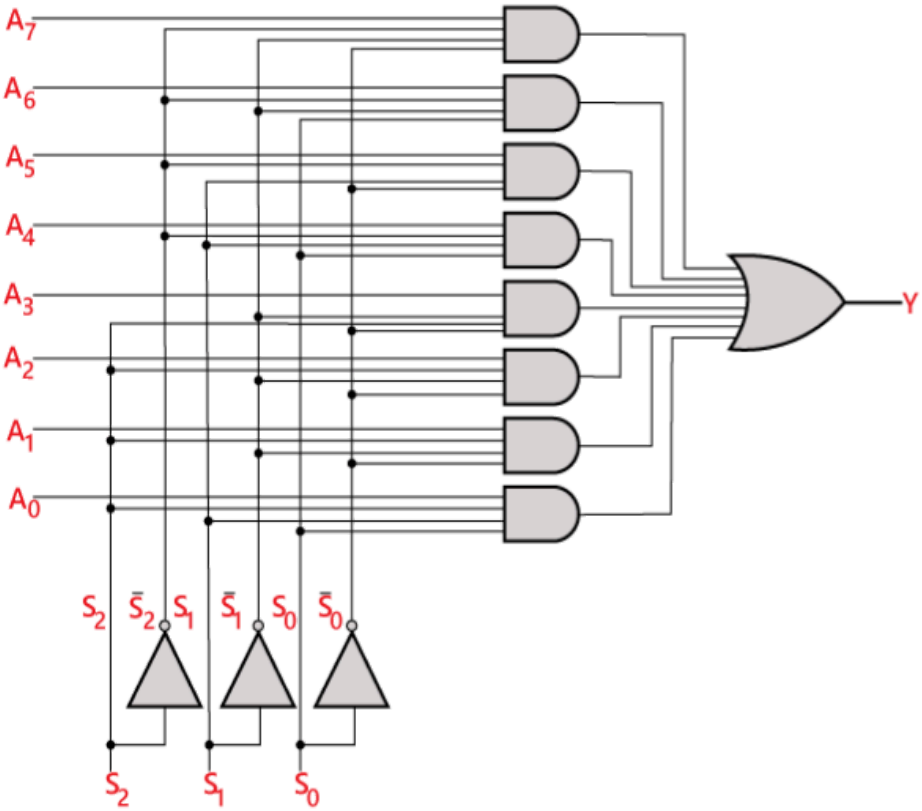
Truth Table:

S_2	S_1	S_0	Y (Output)
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7



Boolean Expression:

$$Y = \overline{S}_2\overline{S}_1\overline{S}_0A_0 + \overline{S}_2\overline{S}_1S_0A_1 + \overline{S}_2S_1\overline{S}_0A_2 + \overline{S}_2S_1S_0A_3 + S_2\overline{S}_1\overline{S}_0A_4 + S_2\overline{S}_1S_0A_5 + S_2S_1\overline{S}_0A_6 + S_2S$$

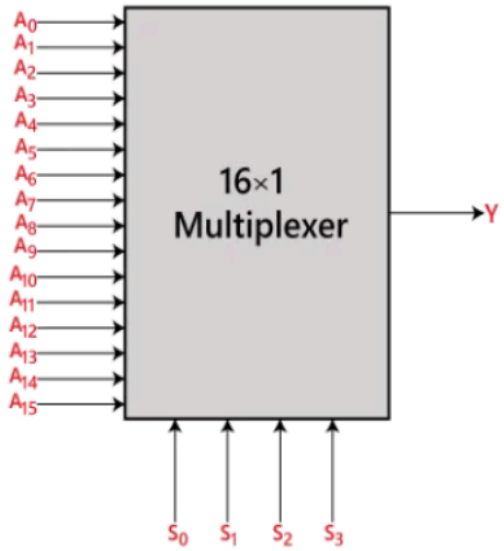

A7
A6
A5
A4
A3
A2
A1
A0
Y
S2
S̄2
S1
S̄1
S0
S̄0
S2
S1
S0

5. The 16x1 Multiplexer

- The 16×1 multiplexer has 16 inputs (A_0 to A_{15}), 4 selection lines (S_3, S_2, S_1, S_0), and a single output (Y).
- Based on the specific combination of inputs present at the selection lines, one of the 16 inputs will be routed to the output.

Truth Table:

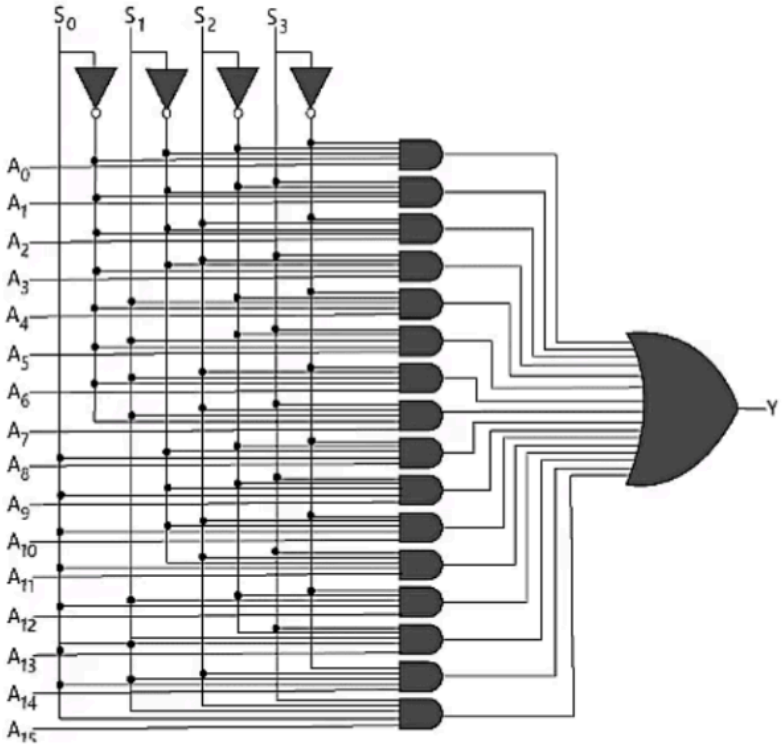
S_3	S_2	S_1	S_0	Y (Output)
0	0	0	0	A_0
0	0	0	1	A_1
0	0	1	0	A_2
0	0	1	1	A_3
0	1	0	0	A_4
0	1	0	1	A_5
0	1	1	0	A_6
0	1	1	1	A_7
1	0	0	0	A_8
1	0	0	1	A_9
1	0	1	0	A_{10}
1	0	1	1	A_{11}
1	1	0	0	A_{12}
1	1	0	1	A_{13}
1	1	1	0	A_{14}
1	1	1	1	A_{15}



16-1 Multiplexer

Boolean Expression:

$$Y = \overline{S}_3\overline{S}_2\overline{S}_1\overline{S}_0A_0 + \overline{S}_3\overline{S}_2\overline{S}_1S_0A_1 + \overline{S}_3\overline{S}_2S_1\overline{S}_0A_2 + \ldots + S_3S_2S_1S_0A_{15}$$



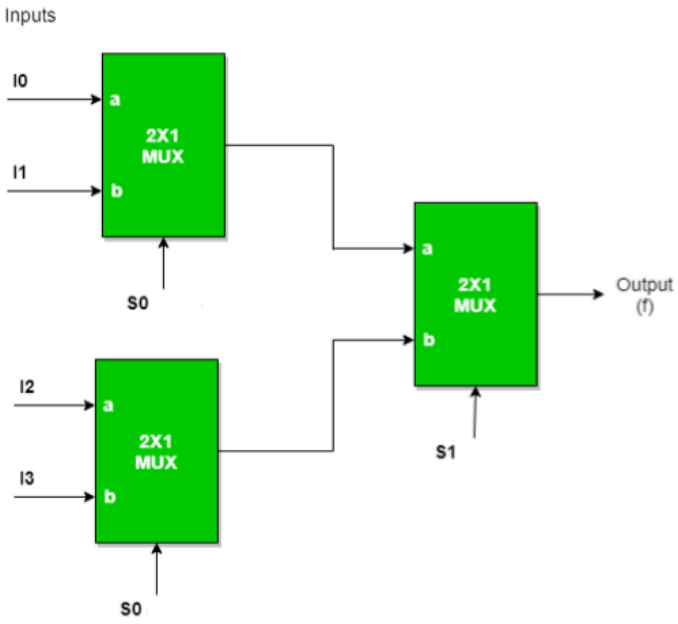
(Expands to cover all 16 minterms associated with their respective data lines).---

6. Multiplexer Implementation (Cascading)

Higher-order multiplexers can be implemented using combinations of lower-order multiplexers. The mathematical approach to finding the number of required MUXes is defined by successive division of data lines.

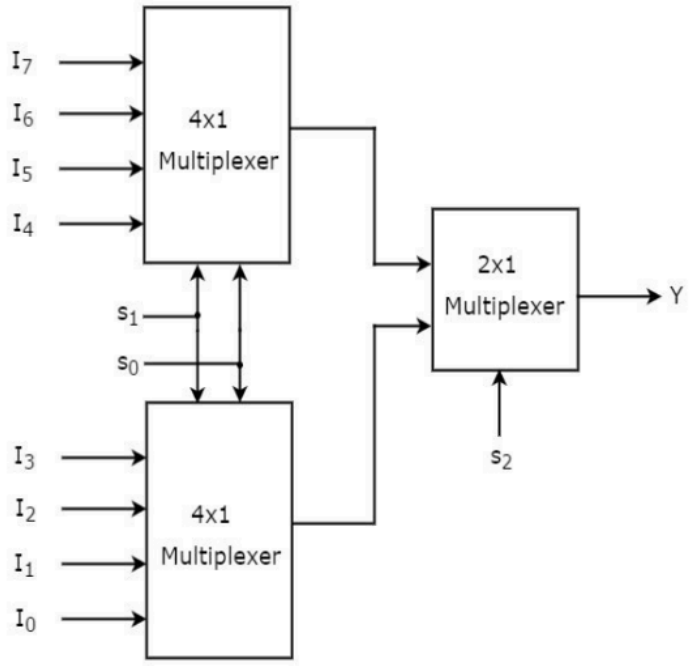
Implementation 1: 4:1 MUX using 2:1 MUX

- **Required MUX:** 4:1 ($n = 4$ data lines)
- **Available MUX:** 2:1 ($p = 2$ data lines)
- **Stage 1 (f):** $n/p = 4/2 = 2$
- **Stage 2 (s):** $f/p = 2/2 = 1$
- **Total 2:1 MUXes Required:** $f + s = 2 + 1 = 3$



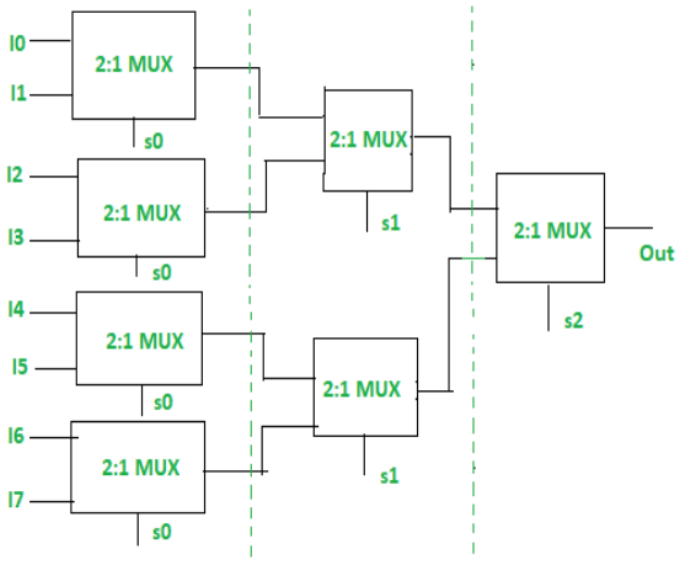
Implementation 2: 8x1 MUX using 4x1 and 2x1 MUX

- We need two 4×1 multiplexers and one 2×1 multiplexer.
- **Stage 1:** For getting 8 data inputs, we need two 4×1 multiplexers (8 / 4 = 2).
- **Stage 2:** The 4×1 multiplexers produce two outputs total. To get the final output, we need one 2×1 multiplexer (2 / 2 = 1).
- **Total Required:** Two 4×1 MUXes and one 2×1 MUX.



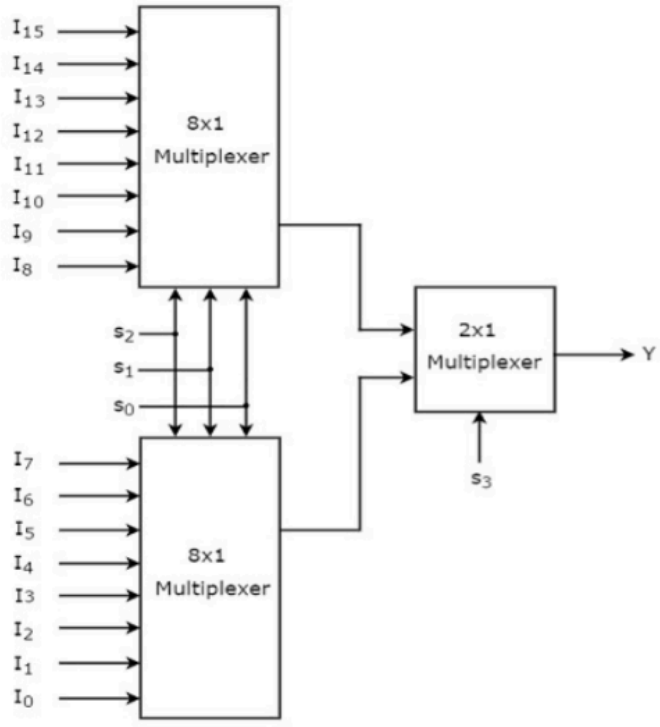
Implementation 3: 8x1 MUX using only 2x1 MUX

- **Required MUX:** 8:1 ($n = 8$ data lines)
- **Available MUX:** 2:1 ($p = 2$ data lines)
- **Stage 1 (f):** $n/p = 8/2 = 4$
- **Stage 2 (s):** $f/p = 4/2 = 2$
- **Stage 3 (t):** $s/p = 2/2 = 1$
- **Total 2:1 MUXes Required:** $f + s + t = 4 + 2 + 1 = 7$



Implementation 4: 16:1 MUX using 8:1 and 2:1 MUX

- **Required MUX:** 16:1 ($n = 16$ data lines)
- **Available MUXes:** 8:1 ($p_1 = 8$) and 2:1 ($p_2 = 2$)
- **Stage 1:** $n/p_1 = 16/8 = 2$ (Requires two 8:1 MUXes)
- **Stage 2:** $f/p_2 = 2/2 = 1$ (Requires one 2:1 MUX)
- **Total Required:** Two 8:1 MUXes and one 2:1 MUX



Implementation 5: 16x1 MUX using only 4x1 MUX

- **Required MUX:** 16:1 ($n = 16$ data lines)
- **Available MUX:** 4:1 ($p = 4$ data lines)
- **Stage 1 (f):** $n/p = 16/4 = 4$
- **Stage 2 (s):** $f/p = 4/4 = 1$
- **Total 4:1 MUXes Required:** $f + s = 4 + 1 = 5$

